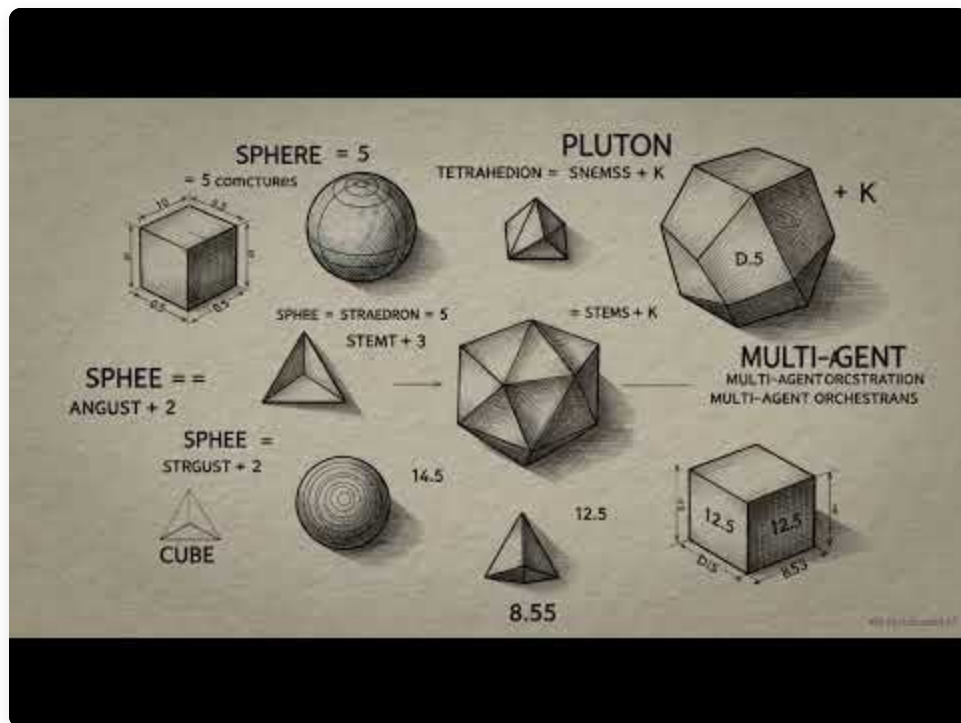


Lesson 1: The Agentic Shift and the Shape of the Cohort

Video Overview



► [Watch on YouTube: Lesson 1 Video Overview](#)

How to use this article

Each major section opens with a **one-line takeaway** drawn directly from the lesson slides. Click *Read more* to expand the deep content — the slide's ecosystem stacks, plus a nested **Further reading** block and a **Socratic study prompt** you can paste into Claude/ChatGPT to talk through the material with an AI tutor. The Socratic prompts are designed to lead you to articulate each principle in your own words, not to feed you definitions.

The Central Thesis

The cohort opens with a single argument: AI engineering has crossed a threshold, and the work that pays now is not the work that paid two years ago. The era of the linear chatbot, a prompt in and a completion out, has collapsed into something more demanding. What sits in front of you is the **Agentic Shift**, the move from request-response systems to self-healing, multi-modal, autonomous agentic systems with measurable ROI.

The program's tagline reflects this: *Bridging the Academic-Enterprise Gap through High-Stakes AI Delivery*. Academic AI courses teach you what a transformer is. Enterprise teams need engineers who can ship an agent that survives a production incident at 2 AM, routes around a failing tool call, observes its own behavior, and stays inside a token budget. That gap is where this cohort plants its flag.

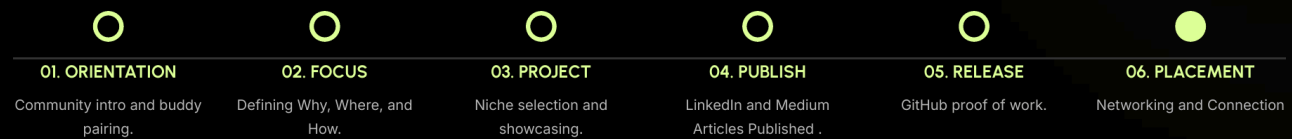
A note on format before the substance: sessions are live and interactive. There are no recordings. If you miss a session, you lean on your buddy pair. This is deliberate. The expectation is that you show up, engage, and treat the cohort as a working community rather than a video library.

The Six-Milestone Roadmap

TL;DR (from slide): A structured journey from community integration to high-impact career placement — six milestones designed to *transform passive learners into active, published experts within a single cohort*.

ROADMAP OVERVIEW

A structured journey from community integration to high-impact career placement.



ROADMAP GOAL: Transforming passive learners into active, published experts within a single cohort.

▼ Read more

The cohort structures the journey into six milestones, each one a deliberate transition rather than a checkpoint: 1. **Orientation** — community introduction and buddy pairing. The buddy is your continuity mechanism for missed material and your first peer-review channel. 2. **Focus** — defining your *Why*, *Where*, and *How*. This is where you stop browsing and start narrowing. 3. **Project** — niche selection and showcasing. You pick a domain, a role, and a problem you will be visible on. 4. **Publish** — LinkedIn and Medium articles. Your work goes external, in writing, with your name on it. 5. **Release** — GitHub proof of work. Code, not slides. Architecturally clean, navigable, dogfoodable. 6. **Placement** — networking and connection. The publishing and the release feed into who can find you and why they would want to. Read the goal carefully. The promise is not "you will learn AI." The promise is that by the end you will be a published practitioner with a public artifact trail. The structure exists to force that outcome.

The Five Foundational Concepts

Each student selects one of five foundational concepts as their subject-matter-expert (SME) niche. These are not survey topics; they are pillars deep enough to anchor a body of work around. Pick one and you commit to going past surface level on it.

| CLAIMING YOUR SPECIALIZED DOMAIN

Identify the technical pillar where you will build your "Proof of Work":



I. Agentic Loops

Passionate exploration of autonomous orchestration, tool-use, and multi-step reasoning systems.



II. The FinOps Stack

Strategic publication on LLM vs. SLM routing, token unit economics, and latency optimization.



III. Dynamic AI Data

Becoming an authority on GraphRAG, Knowledge Systems, and the next wave of relational AI memory.



IV. Multimodal Reality

Publishing insights on AI interaction with the physical/digital world through vision and spatial data.



V. Engineering Rigor

Building passion for scientific trust through observability, tracing, and deterministic evaluation.

I. Agentic Loops

TL;DR (from slide): Transforms AI from a passive responder into an active problem-solver capable of multi-step execution and self-correction. *Key takeaway: mastering the balance between autonomous action and verified state management.*

▼ Read more

The core pattern: ****Plan → Act → Observe → Refine.**** This is the orchestration logic that turns an LLM from a text generator into something that can complete a task across many steps. Concretely, this is the territory of ReAct-style reasoning, orchestrator-worker patterns, tool-using agents that decide when to call which function, and multi-step plans that adapt when a step fails. An Agentic Loops specialist understands how to design the planner, what to put in scratch memory, how to structure tool descriptions so the model picks correctly, and how to recover when an Act step returns garbage. The hard problems here are not "can the model use a tool?" but "can the system survive ten tool calls in sequence with one of them

failing silently?" ![Agentic Loops — cross-cloud stacks and protocols]

(images/concept-agentic-loops-3.png) The slide names every major surface area: -

Cross-cloud stacks: Google ADK + Vertex Extensions + Firestore State; AWS

Bedrock Agents + Action Groups + Step Functions; Azure Semantic Kernel + AI

Agent Service; OpenAI Assistants API + Swarm + Function Calling; Anthropic

Computer Use API + Tool Use Hooks. - **Common cross-cloud frameworks:**

LangGraph, CrewAI, AutoGen, PydanticAI. - **Open protocols & libraries:** Model

Context Protocol (MCP), Browser-use, Composio.

▼ Further reading

- **Anthropic — Building Effective Agents** (2024 essay) — opinionated taxonomy of five orchestration patterns (prompt chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer). The vocabulary the field is converging on. → [anthropic.com/research/building-effective-agents] (<https://www.anthropic.com/research/building-effective-agents>) - **ReAct paper** (Yao et al., 2022) — foundational Reason + Act prompting pattern that almost every modern agent loop descends from. → [arxiv.org/abs/2210.03629] (<https://arxiv.org/abs/2210.03629>) - **LangGraph tutorial** — hands-on agentic loop with explicit state graphs (matches the slide's "verified state management" takeaway). → [langchain-ai.github.io/langgraph/tutorials] (<https://langchain-ai.github.io/langgraph/tutorials/>) - **Model Context Protocol (MCP) spec** — Anthropic's open protocol for tool/context interop, named on the slide. → [modelcontextprotocol.io] (<https://modelcontextprotocol.io/>) - **Anthropic Tool Use docs** — practical reference for the tool-use hooks called out on the slide. → [docs.anthropic.com/.../tool-use] (<https://docs.anthropic.com/en/docs/build-with-claude/tool-use/overview>)

▼ Socratic study prompt

Copy this into Claude or ChatGPT, replacing the marked block with the Agentic Loops content from the *Read more* section above:

I'm studying Agentic Loops as part of an AI engineering cohort. Below is the lesson material I'm working from. Engage me in a

Socratic conversation grounded ONLY in this material – do not draw on outside knowledge unless I explicitly ask you to.

--- LESSON MATERIAL ---

[PASTE THE AGENTIC LOOPS SECTION HERE]

--- END ---

Goal of our conversation: lead me to articulate, in my own words, the core principle named in the lesson – "mastering the balance between autonomous action and verified state management."

Rules:

- Ask me one question at a time
- Lead me toward naming the principle myself; don't tell me
- Never give me the answer unless I explicitly ask for it
- If I'm circling, ask a more pointed question that nudges me toward the principle
- Once I've named the principle in my own words, the conversation can flow wherever curiosity takes it – that's gravy

Start with your first question.

II. The FinOps Stack

TL;DR (from slide): Optimizes unit economics by routing tasks based on complexity, preventing over-spending compute on simple queries. *Key takeaway: strategic routing ensures high-reasoning for LLMs and low-cost / latency for SLMs.*

▼ Read more

Token economics is the silent killer of AI products. The FinOps stack covers everything that makes an AI system financially viable at scale: choosing between a frontier LLM and a small specialized model (SLM) per call, prompt caching to avoid re-paying for the same context, model routing to send easy queries to cheap

models and hard queries to expensive ones, batching to amortize fixed costs, and latency optimization because slow agents lose users regardless of how clever they are. A FinOps specialist can look at a system architecture and tell you the cost-per-task, where the spend is concentrated, and which calls are over-provisioned. This is the discipline that separates a demo from a sustainable product. ![FinOps Stack — routing and SLM stacks across providers](images/concept-finops-4.png) The slide names every major surface area: - **Cross-cloud stacks:** Google Model Garden + Provisioned Throughput; AWS Bedrock Eval + Cost Controls; Azure Model Catalog + PTU Scaling; OpenAI GPT-4o mini + Batch API (50% cost); Anthropic Prompt Caching + Haiku 3.5. - **Common router stacks:** Martian, RouteLLM, LiteLLM, NotDiamond. - **SLM mastery:** Gemma 2, Phi-4, Llama 3.2 (1B/3B), vLLM.

▼ Further reading

- **Anthropic prompt-caching docs** — operational; how to actually cut your token bill on repeated context. Named on the slide. → [\[docs.anthropic.com/.../prompt-caching\]](https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching)
(<https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching>) - **FrugalGPT paper** (Stanford, 2023) — foundational work on model cascading, the conceptual base for every router on the slide. → [\[arxiv.org/abs/2305.05176\]](https://arxiv.org/abs/2305.05176)(<https://arxiv.org/abs/2305.05176>) - **RouteLLM paper** (LMSYS, 2024) — open-source router benchmarked across cost-quality trade-offs. Named on the slide. → [\[arxiv.org/abs/2406.18665\]](https://arxiv.org/abs/2406.18665)
(<https://arxiv.org/abs/2406.18665>) - **vLLM docs** — high-throughput LLM serving runtime, named in the slide's SLM mastery section. → [\[docs.vllm.ai\]](https://docs.vllm.ai/)
(<https://docs.vllm.ai/>) - **LiteLLM docs** — unified routing across 100+ providers, named on the slide. → [\[docs.litellm.ai\]](https://docs.litellm.ai/)(<https://docs.litellm.ai/>)

▼ Socratic study prompt

I'm studying The FinOps Stack as part of an AI engineering cohort
Below is the lesson material I'm working from. Engage me in a Socratic conversation grounded ONLY in this material — do not draw on outside knowledge unless I explicitly ask you to.

--- LESSON MATERIAL ---

[PASTE THE FINOPS STACK SECTION HERE]

--- END ---

Goal of our conversation: lead me to articulate, in my own words, the core principle named in the lesson – "strategic routing ensure high-reasoning for LLMs and low-cost / latency for SLMs."

Rules:

- Ask me one question at a time
- Lead me toward naming the principle myself; don't tell me
- Never give me the answer unless I explicitly ask for it
- If I'm circling, ask a more pointed question that nudges me toward the principle
- Once I've named the principle in my own words, the conversation can flow wherever curiosity takes it – that's gravy

Start with your first question.

III. Dynamic AI Data

TL;DR (from slide): Transitions from basic keyword search to relational Knowledge Systems where AI understands the web of enterprise entities. *Key takeaway: moving from simple "Search" to deep relational "Knowledge Systems."*

▼ Read more

This is the data layer for agents that need memory and structured knowledge.

****GraphRAG**** is the headline term: rather than chunking documents and retrieving by vector similarity alone, you build a knowledge graph over the source material, so retrieval can traverse relationships, not just match embeddings.

Entities, relations, and the paths between them become first-class. This pillar also covers relational AI memory more broadly: how an agent remembers what happened across sessions, how it represents user-specific context, and how

knowledge systems are kept fresh. The technical question this pillar answers: when retrieval-augmented generation is not enough, what comes next? ![Dynamic AI Data — knowledge graph stacks and lineage tools](images/concept-dynamic-data-5.png) The slide names every major surface area: - **Cross-cloud stacks:** Google Vertex Vector Search + BigQuery ML; AWS Amazon Neptune (Graph) + Kendra; Azure AI Search (Hybrid) + Cosmos DB; OpenAI Vector Stores API + File Search; Anthropic Contextual Retrieval patterns. - **Knowledge graph stacks:** Neo4j, Microsoft GraphRAG, FalkorDB, ArangoDB. - **Data versioning & lineage:** lakeFS, DVC (Data Version Control), W&B Artifacts.

▼ Further reading

- **Microsoft GraphRAG repo + paper** (2024) — the canonical GraphRAG reference implementation, named on the slide. → [\[github.com/microsoft/graphrag\]](https://github.com/microsoft/graphrag)(<https://github.com/microsoft/graphrag>) - **Anthropic — Contextual Retrieval** (Sept 2024) — practical technique that prepends a per-chunk context summary before embedding. 35-67% reduction in retrieval failures. Named on the slide. → [\[anthropic.com/news/contextual-retrieval\]](https://www.anthropic.com/news/contextual-retrieval)(<https://www.anthropic.com/news/contextual-retrieval>) - **Neo4j — RAG patterns guide** — knowledge-graph-backed retrieval, the named lib on the slide. → [\[neo4j.com/developer-blog/knowledge-graph-rag-application/\]](https://neo4j.com/developer-blog/knowledge-graph-rag-application/)(<https://neo4j.com/developer-blog/knowledge-graph-rag-application/>) - **Lost in the Middle** (Liu et al., Stanford, 2023) — foundational read on why naive long-context RAG fails. → [\[arxiv.org/abs/2307.03172\]](https://arxiv.org/abs/2307.03172)(<https://arxiv.org/abs/2307.03172>)

▼ Socratic study prompt

I'm studying Dynamic AI Data as part of an AI engineering cohort. Below is the lesson material I'm working from. Engage me in a Socratic conversation grounded ONLY in this material — do not draw on outside knowledge unless I explicitly ask you to.

--- LESSON MATERIAL ---

[PASTE THE DYNAMIC AI DATA SECTION HERE]

--- END ---

Goal of our conversation: lead me to articulate, in my own words, the core principle named in the lesson – "moving from simple 'Sea to deep relational 'Knowledge Systems.'"

Rules:

- Ask me one question at a time
- Lead me toward naming the principle myself; don't tell me
- Never give me the answer unless I explicitly ask for it
- If I'm circling, ask a more pointed question that nudges me toward the principle
- Once I've named the principle in my own words, the conversation can flow wherever curiosity takes it – that's gravy

Start with your first question.

IV. Multimodal Reality

TL;DR (from slide): Enables AI to interact with the physical and digital world by processing video, images, and spatial data for real-world automation. *Key takeaway: interacting with the physical, analog, and digital world in real time.*

▼ Read more

Pixels, video, spatial data, and the bridge between physical and digital. Multimodal models are no longer a curiosity; they are how AI systems perceive the world the user actually lives in. A specialist here works with vision-language models, video understanding, spatial reasoning over 3D or AR contexts, and the engineering that lets an agent both see and act. The interesting frontier is not single-frame image understanding; it is sustained multimodal interaction, where an agent watches a stream, maintains state about what it has seen, and grounds its actions in the physical environment. ![\[Multimodal Reality — vision and action stacks\]](#)
(images/concept-multimodal-6.png) The slide names every major surface area: -

Cross-cloud stacks: Google Gemini 1.5 Pro + Vertex AI Vision API; AWS Amazon Rekognition + Multimodal Titan; Azure AI Vision + GPT-4o Vision; OpenAI o1 Vision + Whisper + DALL-E; Anthropic Claude 3.5 Sonnet Vision API. - **Spatial & vision stacks:** OpenCV, Segment Anything (SAM), PyTorch Video. - **Action & digital interaction:** Playwright, WebVoyager, Computer Use SDK.

▼ Further reading

- **Anthropic — Computer Use** announcement (Oct 2024) — sets the frontier for "agent uses a screen." The Computer Use SDK is named on the slide. → [[anthropic.com/news/3-5-models-and-computer-use](https://www.anthropic.com/news/3-5-models-and-computer-use)] (<https://www.anthropic.com/news/3-5-models-and-computer-use>) - **Segment Anything (SAM) paper** (Meta, 2023) — vision foundation model that rewrote pixel-level perception, named on the slide. → [arxiv.org/abs/2304.02643] (<https://arxiv.org/abs/2304.02643>) - **WebVoyager paper** (2024) — agent that navigates real websites end-to-end, named on the slide. → [arxiv.org/abs/2401.13919] (<https://arxiv.org/abs/2401.13919>) - **Gemini 1.5 technical report** — long-context multimodal benchmarks for the model named on the slide. → [arxiv.org/abs/2403.05530] (<https://arxiv.org/abs/2403.05530>)

▼ Socratic study prompt

I'm studying Multimodal Reality as part of an AI engineering cohort. Below is the lesson material I'm working from. Engage me in a Socratic conversation grounded ONLY in this material — do not draw on outside knowledge unless I explicitly ask you to.

--- LESSON MATERIAL ---

[PASTE THE MULTIMODAL REALITY SECTION HERE]

--- END ---

Goal of our conversation: lead me to articulate, in my own words, the core principle named in the lesson — "interacting with the physical, analog, and digital world in real time."

Rules:

- Ask me one question at a time
- Lead me toward naming the principle myself; don't tell me
- Never give me the answer unless I explicitly ask for it
- If I'm circling, ask a more pointed question that nudges me toward the principle
- Once I've named the principle in my own words, the conversation can flow wherever curiosity takes it – that's gravy

Start with your first question.

V. Engineering Rigor

TL;DR (from slide): Replaces "vibes-based" testing with scientific metrics to ensure non-deterministic systems are safe, reliable, and production-ready. *Key takeaway: building scientific trust and debugging for non-deterministic systems.*

▼ Read more

The least glamorous and arguably the most valuable. This pillar is observability, tracing, and deterministic evaluation. Concretely: building eval harnesses that catch regressions before deploy, designing deterministic test sets that survive model upgrades, instrumenting traces so you can replay an agent run and see every tool call and intermediate thought, and setting up dashboards that surface drift before users notice. The engineers who own this pillar are the ones who keep AI systems trustworthy in production. Without them, every agent is a vibe-based system held together by hope. ![Engineering Rigor — observability and eval stacks]

(images/concept-rigor-7.png) The slide names every major surface area: - **Cross-cloud stacks:** Google Vertex AI Rapid Evaluation; AWS SageMaker Monitor + Bedrock Guardrails; Azure AI Content Safety + Eval SDK; OpenAI Evals + Moderation API; Anthropic Constitutional AI + System Prompts. - **Observability & tracing:** LangSmith, Arize Phoenix, W&B Prompts. - **Security & reliability evals:** Giskard, PyRIT, DeepEval, RAGAS.

▼ Further reading

- **Hamel Husain — *Your AI Product Needs Evals*** (2024) — industry-standard practical writeup on building eval harnesses. → [\[hamel.dev/blog/posts/evals/\]](https://hamel.dev/blog/posts/evals/)(<https://hamel.dev/blog/posts/evals/>) - **OpenAI Evals repo** — runnable eval harness, named on the slide. → [\[github.com/openai/evals\]](https://github.com/openai/evals)(<https://github.com/openai/evals>) - **Constitutional AI paper** (Anthropic, 2022) — the technique behind Anthropic's safety stack, named on the slide. → [\[arxiv.org/abs/2212.08073\]](https://arxiv.org/abs/2212.08073)(<https://arxiv.org/abs/2212.08073>) - **RAGAS docs** — RAG evaluation framework, named on the slide. → [\[docs.ragas.io\]](https://docs.ragas.io/)(<https://docs.ragas.io/>) - **LangSmith docs** — observability + tracing for LLM apps, named on the slide. → [\[docs.smith.langchain.com\]](https://docs.smith.langchain.com/)(<https://docs.smith.langchain.com/>)

▼ Socratic study prompt

I'm studying Engineering Rigor as part of an AI engineering cohort. Below is the lesson material I'm working from. Engage me in a Socratic conversation grounded ONLY in this material — do not draw on outside knowledge unless I explicitly ask you to.

--- LESSON MATERIAL ---

[PASTE THE ENGINEERING RIGOR SECTION HERE]

--- END ---

Goal of our conversation: lead me to articulate, in my own words, the core principle named in the lesson — "building scientific trust and debugging for non-deterministic systems."

Rules:

- Ask me one question at a time
- Lead me toward naming the principle myself; don't tell me
- Never give me the answer unless I explicitly ask for it
- If I'm circling, ask a more pointed question that nudges me

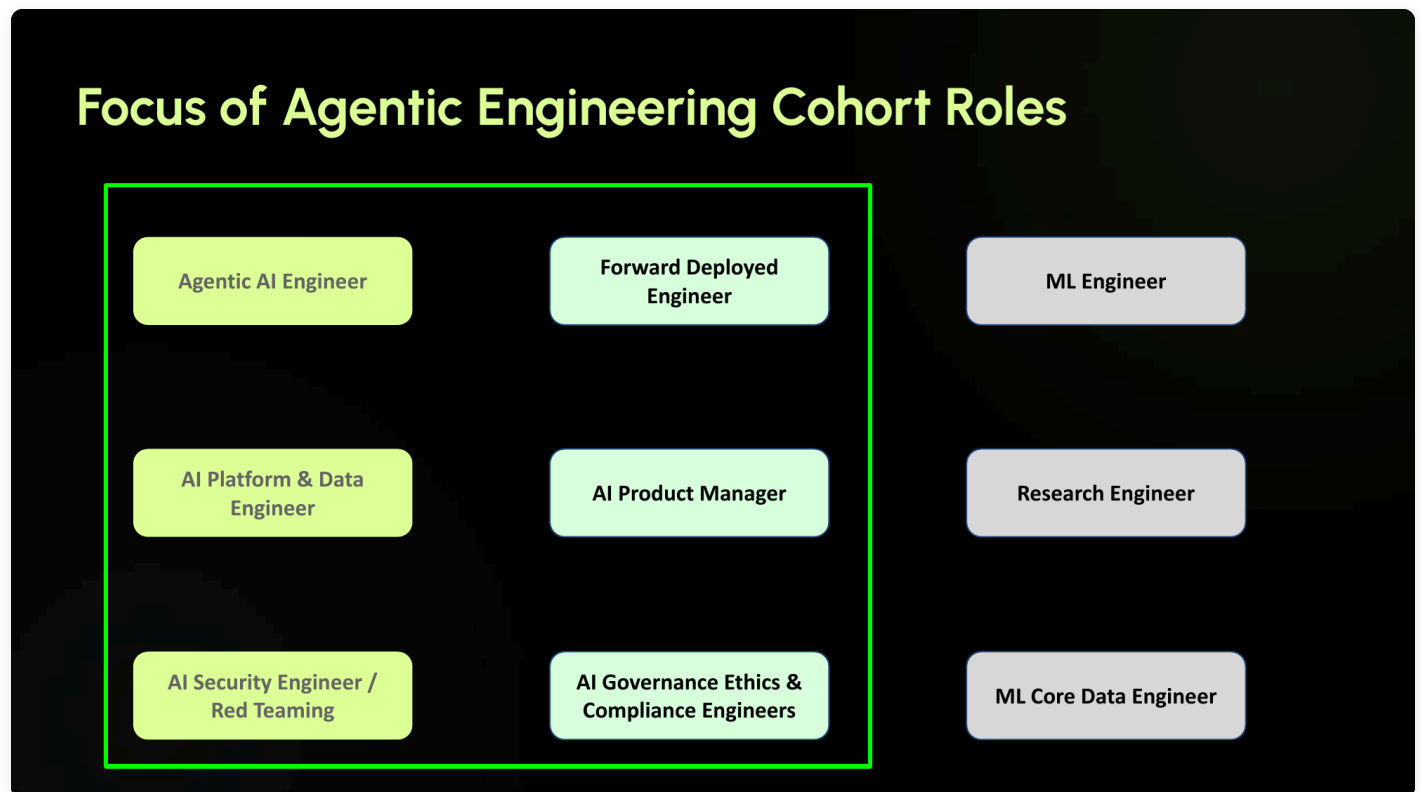
toward the principle

- Once I've named the principle in my own words, the conversation can flow wherever curiosity takes it – that's gravy

Start with your first question.

The Six AI Engineering Roles

Alongside the foundational concept, each student picks a North Star role from six in-scope tracks. The role is the career identity you build the cohort around. The lesson made the in-scope versus out-of-scope split very visible:



Agentic AI Engineer

TL;DR (from slide): Focuses on autonomous systems that use tools and reason through multi-step tasks.

▼ Read more

Slide-named sub-competencies: - **Orchestration Frameworks** — mastery of LangGraph, Agent Development Kit, CrewAI, or AutoGen. - **Reasoning Patterns** — implementation of Chain-of-Thought (CoT) and ReAct prompting techniques. - **Tool-Use Design** — creating robust APIs that LLMs can reliably interface with for action execution. - **Evaluation** — building "LLM-as-a-judge" frameworks to test and score agentic reliability.

▼ Further reading

- **Anthropic — Building Effective Agents** — the canonical orchestration-pattern essay. → [anthropic.com/research/building-effective-agents] (<https://www.anthropic.com/research/building-effective-agents>) - **ReAct paper** (Yao et al., 2022) — named in the slide's reasoning-patterns sub-competency. → [arxiv.org/abs/2210.03629] (<https://arxiv.org/abs/2210.03629>) - **Chain-of-Thought paper** (Wei et al., 2022) — named in the slide's reasoning-patterns sub-competency. → [arxiv.org/abs/2201.11903] (<https://arxiv.org/abs/2201.11903>) - **LangGraph tutorial** — named in the slide's orchestration-frameworks sub-competency. → [langchain-ai.github.io/langgraph/tutorials] (<https://langchain-ai.github.io/langgraph/tutorials/>)

▼ Socratic study prompt

I'm exploring whether the Agentic AI Engineer role is right for me. Below is the role definition from the cohort. Engage me in a Socratic conversation grounded ONLY in this material — do not draw on outside knowledge unless I explicitly ask you to.

--- LESSON MATERIAL ---

[PASTE THE AGENTIC AI ENGINEER SECTION HERE]

--- END ---

Goal of our conversation: lead me to articulate, in my own words,

the core focus of this role – "autonomous systems that use tools & reason through multi-step tasks" – and decide whether my own strength match it.

Rules:

- Ask me one question at a time
- Lead me toward naming the role's core focus myself; don't tell me
- Never give me the answer unless I explicitly ask for it
- If I'm circling, ask a more pointed question that nudges me toward the focus
- Once I've named it in my own words, push toward the fit question: what skills do I have that align, what's missing
- Beyond that, conversation can flow – that's gravy

Start with your first question.

Forward Deployed Engineer

TL;DR (from slide): The bridge between high-level AI research and specific customer / industry implementations.

▼ Read more

Slide-named sub-competencies: - **Solution Architecting** — rapidly prototyping RAG systems for niche domains like Legal or MedTech. - **Edge AI** — deploying models on-prem or on-device where data cannot leave the perimeter. - **Integration Engineering** — writing custom connectors for legacy ERP / CRM systems to empower AI agents. - **User Feedback Loops** — implementing "Human-in-the-Loop" (HITL) systems for continuous model refinement.

▼ Further reading

- **Palantir — FDE: A Day in the Life** — the canonical writeup on the Forward Deployed Engineer archetype Palantir invented. → [blog.palantir.com] (<https://blog.palantir.com/>) (search "Forward Deployed Engineer") - **Replit** —

How we built Replit Agent** — concrete deployment war stories from a real customer-facing AI build. → blog.replit.com -
****Anthropic — *Claude for Enterprise***** — enterprise integration patterns relevant to FDE work. → [anthropic.com/enterprise](https://www.anthropic.com/enterprise)

▼ Socratic study prompt

I'm exploring whether the Forward Deployed Engineer role is right for me. Below is the role definition from the cohort. Engage me in a Socratic conversation grounded ONLY in this material.

--- LESSON MATERIAL ---

[PASTE THE FORWARD DEPLOYED ENGINEER SECTION HERE]

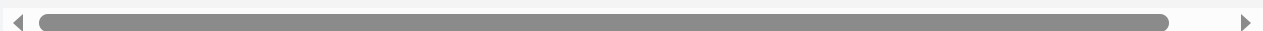
--- END ---

Goal of our conversation: lead me to articulate, in my own words, the core focus of this role – "the bridge between high-level AI research and specific customer / industry implementations" – and decide whether my own strengths match it.

Rules:

- Ask me one question at a time
- Lead me toward naming the role's core focus myself; don't tell me
- Never give me the answer unless I explicitly ask for it
- If I'm circling, ask a more pointed question that nudges me toward the focus
- Once I've named it in my own words, push toward the fit question
- Beyond that, conversation can flow – that's gravy

Start with your first question.



AI Platform & Data Engineer

TL;DR (from slide): Builds the "foundry" where AI models are developed, optimized, and deployed.

▼ Read more

Slide-named sub-competencies: - **Vector Database Management** — optimizing Pinecone, Milvus, or Weaviate at scale for high-retrieval performance. - **Inference Optimization** — deep knowledge of vLLM, TGI, and Quantization techniques (GGUF / AWQ). - **LLMOps** — automating the lifecycle of foundation models and specialized fine-tuned variants. - **GPU Orchestration** — managing high-density compute clusters via Kubernetes (Kueue / Ray).

▼ Further reading

- **Chip Huyen — Designing Machine Learning Systems** (book + blog) — the standard reference for production ML infra. → huyenchip.com/books/ - **vLLM paper + docs** — high-throughput serving runtime, named in the slide's inference-optimization sub-competency. → docs.vllm.ai - **Pinecone — RAG at Scale** — vector-DB scaling guide, names the lib on the slide. → [pinecone.io/learn](https://www.pinecone.io/learn/) - **Ray — Ray for ML Workloads** — GPU orchestration framework, named on the slide. → docs.ray.io

▼ Socratic study prompt

I'm exploring whether the AI Platform & Data Engineer role is right for me. Below is the role definition from the cohort. Engage me in a Socratic conversation grounded ONLY in this material.

--- LESSON MATERIAL ---

[PASTE THE AI PLATFORM & DATA ENGINEER SECTION HERE]

--- END ---

Goal of our conversation: lead me to articulate, in my own words, the core focus of this role – building the "foundry" where AI models are developed, optimized, and deployed – and decide whether my strengths match it.

Rules:

- Ask me one question at a time
- Lead me toward naming the role's core focus myself; don't tell me
- Never give me the answer unless I explicitly ask for it
- If I'm circling, ask a more pointed question that nudges me toward the focus
- Once I've named it in my own words, push toward the final question
- Beyond that, conversation can flow – that's gravy

Start with your first question.

AI Product Manager

TL;DR (from slide): Translates "what is technically possible" into "what is commercially valuable."

▼ Read more

Slide-named sub-competencies: - **AI Unit Economics** — managing token costs vs. latency vs. accuracy trade-offs for ROI. - **UX for Uncertainty** — designing interfaces that handle hallucinations and probabilistic outputs gracefully. - **Data Strategy** — identifying "moat-building" proprietary datasets essential for fine-tuning. - **Compliance Literacy** — navigating the EU AI Act and evolving local regulatory frameworks.

▼ Further reading

- **Hamel Husain — Your AI Product Needs Evals** — eval-driven product thinking; the cost / quality framing applies directly. → [\[hamel.dev/blog/posts/evals/\]\(https://hamel.dev/blog/posts/evals/\)](https://hamel.dev/blog/posts/evals/) - **a16z** —

Emerging Architectures for LLM Applications** — reference architecture you'll be making product trade-offs against. → [a16z.com/emerging-architectures-for-llm-applications/](<https://a16z.com/emerging-architectures-for-llm-applications/>) - ****Lenny's Newsletter — AI PM coverage**** — interviews with AI product leaders; useful for the PM craft side. → [[lennysnewsletter.com](https://www.lennysnewsletter.com/)](<https://www.lennysnewsletter.com/>) - ****EU AI Act primer**** — the regulatory framework named on the slide. → [artificialintelligenceact.eu](<https://artificialintelligenceact.eu/>)

▼ Socratic study prompt

I'm exploring whether the AI Product Manager role is right for me
Below is the role definition from the cohort. Engage me in a
Socratic conversation grounded ONLY in this material.

--- LESSON MATERIAL ---

[PASTE THE AI PRODUCT MANAGER SECTION HERE]

--- END ---

Goal of our conversation: lead me to articulate, in my own words,
the core focus of this role – "translating what is technically
possible into what is commercially valuable" – and decide whether
my strengths match it.

Rules:

- Ask me one question at a time
- Lead me toward naming the role's core focus myself; don't tell me
- Never give me the answer unless I explicitly ask for it
- If I'm circling, ask a more pointed question that nudges me toward the focus
- Once I've named it in my own words, push toward the fit question
- Beyond that, conversation can flow – that's gravy

Start with your first question.

AI Security Engineer / Red Teaming

TL;DR (from slide): Protects the model weights, architecture, and sensitive data from adversarial attacks.

▼ Read more

Slide-named sub-competencies: - **Adversarial Robustness** — defending against prompt injection, jailbreaking, and extraction attacks. - **Privacy-Preserving ML** — implementing Differential Privacy and Federated Learning protocols. - **Red Teaming** — automated "red teaming" of LLM outputs for toxicity or data leakage risks. - **Supply Chain Security** — auditing model weights and third-party dependency vulnerabilities.

▼ Further reading

- **Simon Willison — Prompt injection series** — the standard for security-in-public writing on LLM attacks; explicitly maps to the slide's Adversarial Robustness sub-competency. → [simonwillison.net/tags/prompt-injection/] (<https://simonwillison.net/tags/prompt-injection/>) - **OWASP Top 10 for LLM Applications** — the canonical list of LLM-specific security risks. → [owasp.org/www-project-top-10-for-large-language-model-applications/] (<https://owasp.org/www-project-top-10-for-large-language-model-applications/>) - **Anthropic — Responsible Scaling Policy** — how a frontier lab thinks about supply-chain and weights security. → [[anthropic.com/news/anthropics-responsible-scaling-policy](https://www.anthropic.com/news/anthropics-responsible-scaling-policy)] (<https://www.anthropic.com/news/anthropics-responsible-scaling-policy>) - **PyRIT** (Microsoft) — automated red-teaming toolkit, named in the cohort's Engineering Rigor stack. → [github.com/Azure/PyRIT] (<https://github.com/Azure/PyRIT>)

▼ Socratic study prompt

I'm exploring whether the AI Security Engineer / Red Teaming role is right for me. Below is the role definition from the cohort. Engage me in a Socratic conversation grounded ONLY in this material:

--- LESSON MATERIAL ---

[PASTE THE AI SECURITY ENGINEER SECTION HERE]

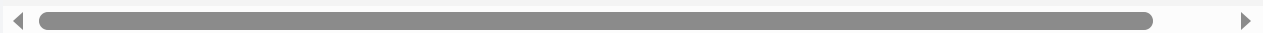
--- END ---

Goal of our conversation: lead me to articulate, in my own words, the core focus of this role – "protecting the model weights, architecture, and sensitive data from adversarial attacks" – and decide whether my strengths match it.

Rules:

- Ask me one question at a time
- Lead me toward naming the role's core focus myself; don't tell me
- Never give me the answer unless I explicitly ask for it
- If I'm circling, ask a more pointed question that nudges me toward the focus
- Once I've named it in my own words, push toward the fit question
- Beyond that, conversation can flow – that's gravy

Start with your first question.



AI Governance, Ethics & Compliance Engineer

TL;DR (from slide): Ensures the AI system is legally sound, morally responsible, and transparent.

▼ Read more

Slide-named sub-competencies: - **Bias Mitigation** — using tools like Fairlearn to audit and correct model disparate impact. - **Explainability (XAI)** — implementing SHAP or LIME for high-stakes decision-making transparency. - **Regulatory Documentation** — maintaining "Model Cards" and detailed transparency logs for audits. - **Policy Implementation** — translating legal requirements into hard-coded technical guardrails.

▼ Further reading

- **NIST AI Risk Management Framework** — the canonical governance reference in the US regulatory environment. → [[nist.gov/itl/ai-risk-management-framework](https://www.nist.gov/itl/ai-risk-management-framework)](<https://www.nist.gov/itl/ai-risk-management-framework>) - **Model Cards paper** (Mitchell et al., 2018) — the artifact named in the slide's regulatory-documentation sub-competency. → [arxiv.org/abs/1810.03993](<https://arxiv.org/abs/1810.03993>) - **EU AI Act primer** — the regulatory framework directly named on the slide. → [artificialintelligenceact.eu/](<https://artificialintelligenceact.eu/>) - **Fairlearn** — bias-mitigation toolkit named on the slide. → [fairlearn.org](<https://fairlearn.org/>)

▼ Socratic study prompt

I'm exploring whether the AI Governance, Ethics & Compliance Engineer role is right for me. Below is the role definition from the cohort. Engage me in a Socratic conversation grounded ONLY in this material.

--- LESSON MATERIAL ---

[PASTE THE AI GOVERNANCE SECTION HERE]

--- END ---

Goal of our conversation: lead me to articulate, in my own words, the core focus of this role – "ensuring the AI system is legally

sound, morally responsible, and transparent" – and decide whether my strengths match it.

Rules:

- Ask me one question at a time
- Lead me toward naming the role's core focus myself; don't tell r
- Never give me the answer unless I explicitly ask for it
- If I'm circling, ask a more pointed question that nudges me toward the focus
- Once I've named it in my own words, push toward the fit question
- Beyond that, conversation can flow – that's gravy

Start with your first question.

What Is Deliberately Out of Scope

Three traditional roles are explicitly excluded from this cohort: **ML Engineer**, **Research Engineer**, and **ML Core Data Engineer**. These are the "build the model from scratch" roles, the world of training pipelines, dataset curation for pretraining, novel architecture research (scaling laws, beyond-Transformer architectures like Mamba), and the petabyte-scale data work that supports it (feature stores, lineage, unstructured pipelines).

The reasoning is strategic, not dismissive. The cohort's thesis is that the leverage in the current AI labor market has shifted. Foundation models are increasingly commoditized at the production layer; the differentiated, high-paying work is in *applying* those models, *orchestrating* them into agentic systems, *deploying* them into real workflows, and *governing* their behavior. Building a foundation model from scratch is a multi-hundred-million-dollar exercise concentrated at a handful of labs. Building the systems on top of those foundation models is where most teams need talent right now. The cohort optimizes for that demand curve.

The Deliverable Triad: LinkedIn, Medium, GitHub

Every student in the cohort ships three categories of public artifact:

| PROGRAM DELIVERY REQUIREMENTS



2–3 LinkedIn Posts: Technical "build-in-public" snippets or niche industry insights that spark professional conversation.



1+ Deep-Dive Medium Article: A long-form technical piece or case study that proves your ability to navigate and explain complexity.



1+ Dedicated GitHub Project: A clean, architecturally sound repository showcasing implementation skills and engineering rigor.

- **2–3 LinkedIn posts** — short, technical, build-in-public snippets. Not motivational quotes. Specific things you learned, broke, or shipped, written for a peer audience.
- **1+ Medium article** — long-form deep dive. A case study, a teardown, an architecture write-up. Something a hiring engineer could read and form a real opinion of your craft.
- **1+ GitHub project** — a clean, architecturally sound repository. Not a notebook dump. Code structured the way you would structure it on a team.

The framing the program uses is direct: **Visibility follows publication. Opportunities follow authority.** You do not become a known engineer in a niche by being good in private. You become known by writing what you know down, in public, repeatedly, in a place searchable by the people who hire.

The default audience for these artifacts is FAANG+ recruiters, but the same artifact strategy serves anyone whose attention you want — investors, design partners, future co-founders, conference programmers. Publishing is not a marketing afterthought

tacked onto a technical program. It is part of the curriculum because the cohort treats career outcomes as a problem to be engineered, not a thing that just happens to people who are good enough.

Published Presence and Passionate Narrative

The lesson framed the personal brand around a deliberate split. **Published Presence** is the unified profile strategy that converts your projects into public artifacts and positions you as a specialized expert legible to recruiters and engineering leaders. **Passionate Narrative** is the storytelling layer on top of the artifacts: continuous discovery, build-in-public snippets, evidence of authentic curiosity. The two work together. Posts and repos without a narrative are just noise; a narrative without artifacts is just claims. Both halves are required.

Week 1 Assignments

Three deliverables anchor Week 1, and they fit together as a unit: pick a role, pick a niche, and start working in the actual tooling.

1. Define Your North Star

Take the six in-scope roles. Analyze each one against your personal traits — what you are good at, what you tolerate, what bores you — and against external market demand. Declare a chosen role. The output is a written rationale, not just a label. The point of the exercise is to force a defensible choice rather than a fuzzy "I think maybe the agentic one."

2. Claim Your Territory

Intersect personal passion with the five foundational concepts. The SME niche is the concept you commit to becoming credibly deep in. Combined with the North Star role, this is the (role, concept) pair that defines what you publish about, what you build, and what you become legible for.

3. Upgrade the Gourmet Party Planner App

The cohort provides a baseline agentic application called *The Gourmet Party Planner*. Your job is to deconstruct it, fine-tune the sub-agent configurations, and in the process master the toolchain — Cursor and AntiGravity. This is the hands-on counterweight to the two strategic assignments above. The strategic ones force you to choose; this one forces you to ship. By the end of the week you should have a working modified agent and the muscle memory of the dev environment you will use for the rest of the cohort.

▼ A worked example prompt (credit: Malik Mokhtar)

To make the assignment concrete, here is one student's prompt for upgrading the baseline app into a multi-agent system called ****AGEP**** (Autonomous Gourmet Event Planner). It is a useful reference for the level of specificity the assignment rewards: explicit roles, explicit temperatures, explicit failure paths, explicit test scenarios. > ****Role:**** Senior AI Platform Engineer > ****Task:**** Develop "AGEP" (Autonomous Gourmet Event Planner) using Google ADK > > Act as a Senior AI Platform Engineer. Build a Python-based multi-agent system named AGEP using the Google Agent Development Kit (ADK). This system must implement a strict Plan-Act-Reflect loop with specialized sub-loops for budget correction and hallucination prevention. > > ****1. System Architecture & Agent Roles**** > Define four distinct agents using ADK abstractions: > - ****ArchitectAgent (The Planner):**** (Temp 0.7) Responsible for strategic decomposition of user intent into a JSON-based menu and task roadmap. > - ****ExecutorAgent (The Act):**** (Temp 0.1) Uses tools (Search, simulated Grocery/Nutrition APIs) to ground the plan in real-world pricing and availability. > - ****CriticAgent (The Reflector):**** (Temp 0.0) Performs logical validation. Triggers the Correction Loop if budget or macro constraints are violated. > - ****VerifierAgent (The Safety):**** (Temp 0.0) A "Zero-Trust" agent. Triggers the Hallucination Prevention Loop by cross-referencing ingredients against a Nutrition/Allergy DB. > > ****2. Logic & Flow Requirements**** > - ****Iterative Loop:**** Implement a controller that allows for a maximum of 5 iterations before failing. > - ****Correction Loop:**** If the CriticAgent rejects the plan (e.g., budget overage), it must pass specific "Delta-Instructions" back to the Architect for a re-plan. > - ****Hallucination Prevention:**** The VerifierAgent must audit every ingredient. If a safety violation is found (e.g., Architect suggesting "almonds" for a nut-allergy guest), the plan is immediately rejected and sent back to the Architect. > -

****Human Escalation:**** Throw a `ConstraintConflictError` if constraints are mathematically impossible (e.g., \$10 budget for 20 people). > > ****3.**

Implementation Details > - ****Language:**** Python 3.10+ > - ****Framework:**** Google Agent Development Kit (ADK) > - ****Code Style:**** Use strict Type Hinting, Clean Architecture (separate files for `agents.py`, `tools.py`, `state.py`, and `main.py`) > - ****Naming Conventions:**** snake_case for functions/variables, PascalCase for classes > - Focus on high readability for students. > > ****4. Test Scenario** > - ****Input:**** "Dinner for 6, \$200 budget, 1 Vegan, 1 Nut-Allergy. Use Salmon & Quinoa." > - ****Expected Behavior:**** System should catch if Salmon prices exceed budget or if a recipe accidentally includes a nut-based filler, and loop until a valid plan is formed. Notice what this prompt does well. It separates strategic planning (Architect, high temperature) from grounded execution (Executor, low temperature) from validation (Critic, zero temperature) from safety (Verifier, zero temperature). It defines the loop's exit conditions, both successful and failure. It specifies a concrete test case the system has to pass, which means the prompt itself encodes its own acceptance criteria. This is the level of design intent the assignment is asking you to bring.

▼ A reference implementation: AGEP

For an example of what the assignment looks like fully built out, see [\[github.com/IrfanThomson/agep\]](https://github.com/IrfanThomson/agep)(<https://github.com/IrfanThomson/agep>) — a four-agent Plan-Act-Reflect dinner-event planner on Google ADK, iterated v1 → v2 (adversarial Saboteur agent) → v3 (post-loop Chef stage producing a cookable plan). The repo includes a 6-minute explainer video walking through the architecture and the loop in action: [\[!\[\[AGEP explainer \(6:39\)\]\]](https://img.youtube.com/vi/Q2C8U8drwak/hqdefault.jpg) (<https://img.youtube.com/vi/Q2C8U8drwak/hqdefault.jpg>) (<https://youtu.be/Q2C8U8drwak>) ► [\[Watch the AGEP explainer \(6:39\)\]](https://youtu.be/Q2C8U8drwak) (<https://youtu.be/Q2C8U8drwak>) Use it the way you would use a worked solution in a math textbook: study the structure, notice the design choices, then build your own version differently.

▼ Bonus exercise: analyze any repo through the five concepts (Socratic)

A useful habit you should form this week: take a real codebase — your own Gourmet Party Planner build, AGEP, or literally any AI repo on GitHub — and dissect

it through the lens of the five Foundational Concepts. The point is to make the concepts **applied**, not memorized. The example below uses `[github.com/IrfanThomson/agep](https://github.com/IrfanThomson/agep)` as the target repo — swap that URL for your own build (or any repo you want to study). Paste this into Claude or ChatGPT:

I'm studying agentic engineering and want to analyze a real project through the lens of the five Foundational Concepts from my cohort:

- I. Agentic Loops — Plan / Act / Observe / Refine. Orchestration, tool-use, multi-step reasoning. Principle: balance autonomous action with verified state management.
- II. The FinOps Stack — LLM vs SLM routing, prompt caching, token economics. Principle: strategic routing — high-reasoning for LLMs, low-cost/latency for SLMs.
- III. Dynamic AI Data — GraphRAG, knowledge systems, relational AI memory. Principle: shift from search to relational knowledge systems.
- IV. Multimodal Reality — vision, video, spatial, screen-action. Principle: AI interacting with the physical, analog, and digital world in real time.
- V. Engineering Rigor — observability, tracing, deterministic evaluation. Principle: replace "vibes-based" testing with scientific evaluation for non-deterministic systems.

Target repo: `https://github.com/IrfanThomson/agep`
(swap this URL for any GitHub repo you want to analyze)

Read through the repo — README, source structure, key files — and engage me in a Socratic conversation about how each of the five concepts is applied, or deliberately not applied, in this project.

Goal: lead me to articulate, in my own words, which concepts dominate the architecture, which are absent, and what the design

consequences of those choices are.

Rules:

- Ask me one question at a time
- Lead me toward the observation myself; don't tell me
- Ground every question in something concrete from the repo (a specific file, function, README claim, or design decision)
- Never give me the answer unless I explicitly ask for it
- If I'm circling, ask a more pointed question that nudges me toward a specific design choice in the repo
- After we've worked through the named concepts, the conversation can flow wherever curiosity takes it – that's gravy

Start by asking me which of the five concepts I'd like to begin with, or which one I think dominates the project.

Run this exercise on your own assignment build at the end of the week, then again on a project you admire. The first run shows you what your code reveals about the concepts you internalized. The second shows you what good looks like.

The three assignments are intentionally interlocking. The role tells you who you are becoming. The niche tells you what you are becoming known for. The app upgrade tells you how the work actually feels. None of them work in isolation.

The Quality Deep Work Framework

The closing framing of Lesson 1 is a three-part operating principle for how to behave during the rest of the cohort.

Own the domain. Identify the global leaders in your chosen focus area. Read what they ship, watch what they say, dissect their systems. Then form your own point of view. Not a regurgitation, a position. Engineers without a POV are interchangeable; engineers with a defensible POV are recruited.

Master the flow. Pick apart multi-agent orchestration in real systems. Find the seams. Understand specifically where hand-offs between agents fail, where context gets lost, where retries paper over real bugs. The people who can articulate these failure modes are the people other teams want in the room when something is broken.

Founder mindset. The orchestration logic *is* the moat for an AI startup. The model is rented; the way you wire it together is yours. And from the FAANG+ angle, the same skill — the ability to articulate architectural trade-offs in agentic systems out loud, in an interview, in a design review — is what gets you hired into senior roles. Whether the goal is a startup or a senior IC seat, the underlying competence is identical: deep, deliberate work on systems that are still being figured out in public.

Where Lesson 1 Leaves You

Lesson 1 is a framing lesson. It does not teach you how to build an agent; it teaches you the frame the cohort uses to decide what is worth building, what is worth publishing, and what is worth becoming known for. The Agentic Shift is the macro thesis. The five concepts and six roles are the menu. The deliverable triad is the output channel. The Week 1 assignments are the on-ramp. The Quality Deep Work framework is the posture you bring to all of it.

The work begins now. By the time you reach the Publish milestone, the choices you made in Week 1 will already be visible in the artifacts you put your name on. Choose accordingly.